

CSS3 @supports Rule

Complete Guide to Feature Detection and Progressive Enhancement

CSS Advanced Techniques

May 10, 2026

Contents

1	Introduction to @supports	2
1.1	What is @supports?	2
1.2	Why Use @supports?	2
1.3	Browser Compatibility	3
2	Basic @supports Syntax	4
2.1	Simple Feature Detection	4
2.2	Detecting Different Property Values	4
2.3	Logical Operators	5
3	Progressive Enhancement Patterns	7
3.1	Pattern 1: CSS Grid with Fallback	7
3.2	Pattern 2: CSS Custom Properties with Fallback	7
3.3	Pattern 3: Modern Features with Graceful Degradation	8
4	Practical Application Examples	10
4.1	Feature Detection for Layout Systems	10
4.2	Feature Detection for Typography	11
4.3	Feature Detection for Visual Effects	11
4.4	Feature Detection for Responsive Design	12
4.5	Complex Condition Patterns	13
4.6	Feature Detection for Selectors	14
5	Real-World Examples	15
5.1	Example 1: Responsive Navigation	15
5.2	Example 2: Modern Card Component	16
5.3	Example 3: Theme with CSS Variables	16
6	Best Practices	18
6.1	Guidelines for Using @supports	18
6.2	Performance Considerations	18
6.3	Accessibility Considerations	18
7	Common Feature Checks	20
7.1	Layout Features	20
7.2	Visual Effects Features	20
7.3	Typography Features	21
7.4	Advanced Features	21

8	Quick Reference	22
8.1	@supports Syntax Cheat Sheet	22
8.2	Progressive Enhancement Template	22
8.3	Multi-Browser Setup	23

Chapter 1

Introduction to @supports

1.1 What is @supports?

The `@supports` CSS at-rule enables feature detection directly in CSS by checking if the browser supports a specific CSS property or property-value pair. It's part of the CSS Conditional Rules Module and enables progressive enhancement.

Key facts about @supports:

- Performs CSS feature detection at parse time
- Works with property-value pairs or selectors
- Part of CSS Conditional Rules Module Level 3
- Enables progressive enhancement without JavaScript
- Allows fallback styles for unsupported features
- Supports complex conditions (and, or, not)
- Modern browser support (all modern browsers)
- Zero performance overhead for unsupported rules

1.2 Why Use @supports?

Progressive enhancement is crucial for modern web development:

- **Browser compatibility:** Safely use cutting-edge CSS features
- **No JavaScript needed:** Feature detection in pure CSS
- **Graceful degradation:** Fallback styles for older browsers
- **Better performance:** Avoid JavaScript detection overhead
- **Cleaner code:** Organize styles by capability
- **Maintainability:** Clear intent of fallback strategies
- **User experience:** Enhanced experience for modern browsers

1.3 Browser Compatibility

The `@supports` rule is supported in modern browsers:

- Chrome 28+ (2013)
- Firefox 22+ (2013)
- Safari 9+ (2015)
- Edge (all versions)
- Opera 12.1+ (2012)
- Internet Explorer 11 (not supported)

Chapter 2

Basic @supports Syntax

2.1 Simple Feature Detection

```
/* Basic @supports syntax */
@supports (property: value) {
    /* Styles applied if browser supports the property */
}

/* Example: Detect CSS Grid support */
@supports (display: grid) {
    .container {
        display: grid;
        grid-template-columns: repeat(3, 1fr);
        gap: 1rem;
    }
}

/* Example: Detect Flexbox support */
@supports (display: flex) {
    .navbar {
        display: flex;
        justify-content: space-between;
    }
}

/* Example: Detect CSS Custom Properties */
@supports (--custom: value) {
    :root {
        --primary-color: #0066cc;
        --spacing: 1rem;
    }
}
```

2.2 Detecting Different Property Values

```
/* Detect specific property values */
@supports (display: flex) {
```

```
    /* Browser supports Flexbox */
}

@supports (display: grid) {
    /* Browser supports CSS Grid */
}

@supports (position: sticky) {
    /* Browser supports position: sticky */
}

@supports (backdrop-filter: blur(10px)) {
    /* Browser supports backdrop filter */
}

@supports (mix-blend-mode: multiply) {
    /* Browser supports blend modes */
}

@supports (-webkit-appearance: none) {
    /* Webkit-based browsers */
}
```

2.3 Logical Operators

```
/* AND operator - all conditions must be true */
@supports (display: flex) and (flex-wrap: wrap) {
    .container {
        display: flex;
        flex-wrap: wrap;
    }
}

/* OR operator - at least one condition must be true */
@supports (display: flex) or (display: grid) {
    .container {
        display: flex;
    }
}

/* NOT operator - negates a condition */
@supports not (display: grid) {
    /* Fallback for browsers without Grid */
    .container {
        display: flex;
    }
}

/* Complex conditions */
@supports (display: grid) and (gap: 1rem) and not (display: -webkit-flex)
{
    .modern-layout {
```

```
        display: grid;
        gap: 1rem;
    }
}
```


Chapter 3

Progressive Enhancement Patterns

3.1 Pattern 1: CSS Grid with Fallback

```
<style>
  /* Fallback for non-Grid browsers */
  .gallery {
    display: flex;
    flex-wrap: wrap;
    gap: 1rem;
  }

  .gallery-item {
    flex: 0 0 calc(50% - 0.5rem);
  }

  /* Modern Grid layout for supporting browsers */
  @supports (display: grid) {
    .gallery {
      display: grid;
      grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
      gap: 1rem;
    }

    .gallery-item {
      flex: none; /* Reset flex properties */
    }
  }
</style>

<div class="gallery">
  <div class="gallery-item">Item 1</div>
  <div class="gallery-item">Item 2</div>
  <div class="gallery-item">Item 3</div>
</div>
```

3.2 Pattern 2: CSS Custom Properties with Fallback

```
/* Fallback colors for browsers without CSS variables */
.button {
  background-color: #0066cc;
  color: white;
  border: 1px solid #004999;
}

/* Modern approach with CSS variables */
@supports (--custom: value) {
  :root {
    --primary-color: #0066cc;
    --primary-dark: #004999;
    --text-light: white;
  }

  .button {
    background-color: var(--primary-color);
    color: var(--text-light);
    border: 1px solid var(--primary-dark);
  }

  .button:hover {
    background-color: var(--primary-dark);
  }
}
```

3.3 Pattern 3: Modern Features with Graceful Degradation

```
/* Fallback: Simple solid background */
.hero {
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
}

/* Enhanced: Backdrop filter for modern browsers */
@supports (backdrop-filter: blur(10px)) {
  .hero-overlay {
    backdrop-filter: blur(10px);
    background: rgba(255, 255, 255, 0.8);
  }
}

/* Fallback: Basic box shadow */
.card {
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
}

/* Enhanced: Drop shadow filter */
@supports (filter: drop-shadow(0 2px 4px rgba(0, 0, 0, 0.2))) {
  .card {
    filter: drop-shadow(0 2px 4px rgba(0, 0, 0, 0.2));
    box-shadow: none;
  }
}
```

```
}  
}
```

Chapter 4

Practical Application Examples

4.1 Feature Detection for Layout Systems

```
<style>
/* Fallback: Float-based layout */
.row {
  margin: 0 -1rem;
  overflow: auto;
}

.col {
  float: left;
  width: 50%;
  padding: 1rem;
}

.col.full {
  width: 100%;
}

/* Modern: CSS Grid layout */
@supports (display: grid) and (gap: 1rem) {
  .row {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 1rem;
    margin: 0;
  }

  .col {
    float: none;
    width: auto;
    padding: 0;
  }

  .col.full {
    width: auto;
    grid-column: 1 / -1;
  }
}
```

```
    }  
</style>  
  
<div class="row">  
  <div class="col">Column 1</div>  
  <div class="col">Column 2</div>  
  <div class="col full">Full Width</div>  
</div>
```

4.2 Feature Detection for Typography

```
/* Fallback: Simple text styling */  
.headline {  
  font-size: 2rem;  
  color: #333;  
}  
  
/* Modern: CSS Text Decoration with advanced options */  
@supports (text-decoration-thickness: 2px) and (text-underline-offset:  
  3px) {  
  .headline {  
    text-decoration: underline;  
    text-decoration-thickness: 2px;  
    text-underline-offset: 3px;  
    text-decoration-color: #0066cc;  
  }  
}  
  
/* Fallback: Regular font smoothing */  
body {  
  -webkit-font-smoothing: antialiased;  
}  
  
/* Modern: Font feature settings */  
@supports (font-feature-settings: "liga" 1) {  
  body {  
    font-feature-settings: "liga" 1;  
  }  
}
```

4.3 Feature Detection for Visual Effects

```
/* Fallback: Simple color background */  
.effect-box {  
  background: #fff;  
  border: 1px solid #ccc;  
}  
  
/* Enhanced: Mix blend modes */  
@supports (mix-blend-mode: multiply) {
```

```
.effect-box {
    background: url('pattern.png');
    mix-blend-mode: multiply;
}

/* Fallback: Regular shadow */
.floating {
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

/* Enhanced: Complex shadow with filter */
@supports (filter: drop-shadow(0 4px 12px rgba(0, 0, 0, 0.15))) {
    .floating {
        filter: drop-shadow(0 4px 12px rgba(0, 0, 0, 0.15));
        box-shadow: none;
    }
}
```

4.4 Feature Detection for Responsive Design

```
/* Fallback: Media queries only */
.container {
    width: 100%;
    max-width: 1200px;
}

@media (max-width: 768px) {
    .container {
        max-width: 100%;
        padding: 0 1rem;
    }
}

/* Modern: CSS Container Queries */
@supports (container-type: inline-size) {
    .card-container {
        container-type: inline-size;
    }

    .card {
        display: grid;
        grid-template-columns: 1fr;
    }

    @container (min-width: 400px) {
        .card {
            grid-template-columns: 1fr 1fr;
        }
    }
}

\end{austlisting}
```

```
\newpage

\chapter{Advanced @supports Patterns}

\section{Detecting Vendor Prefixes}

\begin{lstlisting}[language=CSS]
/* Detect WebKit-specific features */
@supports (-webkit-appearance: none) {
    input {
        -webkit-appearance: none;
        width: 100%;
    }
}

/* Detect Firefox-specific features */
@supports (-moz-user-select: none) {
    .no-select {
        -moz-user-select: none;
    }
}

/* Detect CSS Mask (Webkit) */
@supports (-webkit-mask-image: url(#mask)) or (mask-image: url(#mask)) {
    .masked {
        mask-image: url(#mask);
    }
}

/* Detect Backdrop Filter (Webkit) */
@supports (-webkit-backdrop-filter: blur(10px)) or (backdrop-filter:
    blur(10px)) {
    .frosted {
        backdrop-filter: blur(10px);
    }
}
}
```

4.5 Complex Condition Patterns

```
/* Modern Grid and Gaps */
@supports (display: grid) and (gap: 1rem) {
    .grid {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
        gap: 1rem;
    }
}

/* Either Flexbox or Grid */
@supports (display: flex) or (display: grid) {
    .modern-layout {
```

```
        display: flex;
        gap: 1rem;
    }
}

/* NOT IE11 (basic feature check) */
@supports (display: grid) and not (display: -ms-grid) {
    .modern-only {
        display: grid;
    }
}

/* Complex: Grid with Custom Properties and Subgrid */
@supports (display: grid) and (--custom: value) and
    (grid-template-columns: subgrid) {
    .subgrid-container {
        display: grid;
        grid-template-columns: subgrid;
    }
}
```

4.6 Feature Detection for Selectors

```
/* Detect :has() selector support */
@supports selector(:has(> :last-child)) {
    .container:has(> :last-child) {
        border-bottom: none;
    }
}

/* Detect :is() selector support */
@supports selector(:is(h1, h2, h3)) {
    :is(h1, h2, h3) {
        margin-top: 1rem;
    }
}

/* Detect :where() selector support */
@supports selector(:where(.btn:hover)) {
    :where(.btn, .link):hover {
        text-decoration: underline;
    }
}

/* Detect :focus-visible support */
@supports selector(:focus-visible) {
    button:focus-visible {
        outline: 2px solid #0066cc;
    }
}
```


Chapter 5

Real-World Examples

5.1 Example 1: Responsive Navigation

```
<style>
  /* Fallback: Horizontal list */
  .navbar {
    display: block;
    overflow: auto;
  }

  .navbar a {
    float: left;
    padding: 1rem;
  }

  /* Modern: Flexbox navigation */
  @supports (display: flex) {
    .navbar {
      display: flex;
      justify-content: space-between;
      align-items: center;
    }

    .navbar a {
      float: none;
      padding: 0.75rem 1.5rem;
    }
  }

  /* Even more modern: CSS Grid */
  @supports (display: grid) and (gap: 1rem) {
    .navbar {
      display: grid;
      grid-template-columns: auto 1fr auto;
      gap: 1rem;
      align-items: center;
    }
  }
</style>
```

```
<nav class="navbar">
  <a href="#">Logo</a>
  <div>
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </div>
  <a href="#">Sign In</a>
</nav>
```

5.2 Example 2: Modern Card Component

```
<style>
  /* Base card style */
  .card {
    border: 1px solid #ddd;
    border-radius: 8px;
    padding: 1.5rem;
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);
  }

  /* Enhanced card with modern CSS features */
  @supports (display: grid) and (gap: 1rem) and (backdrop-filter:
    blur(10px)) {
    .card {
      display: grid;
      gap: 1rem;
      background: rgba(255, 255, 255, 0.95);
      backdrop-filter: blur(10px);
    }

    .card-header {
      border-bottom: 1px solid rgba(0, 0, 0, 0.1);
      padding-bottom: 1rem;
      margin-bottom: 0;
    }
  }
</style>

<div class="card">
  <div class="card-header">
    <h3>Card Title</h3>
  </div>
  <p>Card content goes here</p>
</div>
```

5.3 Example 3: Theme with CSS Variables

```
<style>
```

```
/* Fallback: Hard-coded theme colors */
:root {
  --primary-color: #0066cc;
  --secondary-color: #667eea;
  --text-color: #333;
}

/* Graceful enhancement */
@supports (--custom: value) {
  :root {
    --primary-color: #0066cc;
    --secondary-color: #667eea;
    --text-color: #333;
    --spacing: 1rem;
    --border-radius: 8px;
  }

  body {
    color: var(--text-color);
    font-size: 1rem;
  }

  button {
    background: var(--primary-color);
    padding: var(--spacing);
    border-radius: var(--border-radius);
  }
}

/* Dark mode for supporting browsers */
@supports (--custom: value) {
  @media (prefers-color-scheme: dark) {
    :root {
      --primary-color: #4a90ff;
      --text-color: #f0f0f0;
    }
  }
}
</style>
```

Chapter 6

Best Practices

6.1 Guidelines for Using @supports

1. **Provide fallbacks first:** Write fallback styles before @supports
2. **Test thoroughly:** Verify functionality in supported and unsupported browsers
3. **Layer features:** Build from basic to advanced capabilities
4. **Keep fallbacks simple:** Make fallbacks work without cutting-edge features
5. **Document intent:** Add comments explaining why features are conditional
6. **Use for progressive enhancement:** Don't use to hide basic functionality
7. **Combine with media queries:** Use both for responsive and feature-aware design

6.2 Performance Considerations

- **Zero overhead:** @supports checks happen at parse time, not runtime
- **No JavaScript needed:** Faster than JavaScript feature detection
- **Efficient selectors:** Use specific selectors to avoid unnecessary parsing
- **No layout shifts:** Apply all CSS at once, preventing reflows

6.3 Accessibility Considerations

```
/* Ensure keyboard accessibility in all browsers */
button:focus {
    outline: 2px solid #0066cc;
}

@supports selector(:focus-visible) {
    button:focus-visible {
        outline: 2px solid #0066cc;
        outline-offset: 2px;
    }
}
```

```
}

/* Respect user motion preferences */
.animation {
  animation: slide 0.5s ease;
}

@media (prefers-reduced-motion: reduce) {
  .animation {
    animation: none;
  }
}

/* Respect color scheme preferences */
body {
  background: white;
  color: #000;
}

@supports (---custom: value) {
  @media (prefers-color-scheme: dark) {
    body {
      background: #000;
      color: white;
    }
  }
}
```

Chapter 7

Common Feature Checks

7.1 Layout Features

```
/* Flexbox */
@supports (display: flex) { }

/* CSS Grid */
@supports (display: grid) { }

/* CSS Grid with gap support */
@supports (display: grid) and (gap: 1rem) { }

/* Position sticky */
@supports (position: sticky) { }

/* Aspect ratio */
@supports (aspect-ratio: 16 / 9) { }
```

7.2 Visual Effects Features

```
/* Backdrop filter */
@supports (backdrop-filter: blur(10px)) { }

/* Mix blend modes */
@supports (mix-blend-mode: multiply) { }

/* CSS Masks */
@supports (mask-image: url(#mask)) { }

/* Filter effects */
@supports (filter: drop-shadow(0 2px 4px)) { }

/* CSS Shapes */
@supports (shape-outside: circle(50%)) { }
```

7.3 Typography Features

```
/* Font feature settings */
@supports (font-feature-settings: "liga" 1) { }

/* Font variation settings */
@supports (font-variation-settings: "wght" 700) { }

/* Text decoration thickness */
@supports (text-decoration-thickness: 2px) { }

/* Text underline offset */
@supports (text-underline-offset: 3px) { }
```

7.4 Advanced Features

```
/* CSS Container Queries */
@supports (container-type: inline-size) { }

/* CSS Scroll Snap */
@supports (scroll-snap-type: x mandatory) { }

/* CSS Logical Properties */
@supports (padding-inline: 1rem) { }

/* CSS Subgrid */
@supports (grid-template-columns: subgrid) { }

/* CSS Color Level 4 */
@supports (color: hwb(194 0% 0%)) { }
```

Chapter 8

Quick Reference

8.1 @supports Syntax Cheat Sheet

```
/* Basic syntax */
@supports (property: value) {
    /* CSS rules */
}

/* With AND operator */
@supports (prop1: val1) and (prop2: val2) {
    /* Both must be supported */
}

/* With OR operator */
@supports (prop1: val1) or (prop2: val2) {
    /* At least one must be supported */
}

/* With NOT operator */
@supports not (prop: val) {
    /* Property must NOT be supported */
}

/* Complex conditions */
@supports (prop1: val1) and (prop2: val2) or (prop3: val3) {
    /* Complex logic */
}

/* Selector detection */
@supports selector(:has(> :last-child)) {
    /* Selector must be supported */
}
```

8.2 Progressive Enhancement Template

```
/* 1. Base styles - works everywhere */
.component {
```



```
    display: block;
    padding: 1rem;
    border: 1px solid #ccc;
}

/* 2. Enhanced with feature detection */
@supports (display: flex) {
    .component {
        display: flex;
        gap: 1rem;
    }
}

/* 3. Even more enhanced */
@supports (display: grid) {
    .component {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
    }
}

/* 4. Modern with custom properties */
@supports (--custom: value) {
    .component {
        gap: var(--spacing, 1rem);
        border-radius: var(--radius, 8px);
    }
}
```

8.3 Multi-Browser Setup

```
/* Fallback for all browsers */
.box {
    background: blue;
    padding: 1rem;
}

/* Standard modern browsers */
@supports (display: flex) {
    .box {
        display: flex;
        align-items: center;
    }
}

/* WebKit browsers (Safari, Chrome) */
@supports (-webkit-appearance: none) {
    .box {
        -webkit-box-align: center;
    }
}
```

```
/* Firefox */
@supports (-moz-user-select: none) {
  .box {
    /* Firefox-specific rules */
  }
}
```